Tema 2: Gestió de Processos

1. Concepte de Procés i el PCB (*Process Control Block*)

Un **procés** és la representació que fa el Sistema Operatiu d'un **programa en execució**. Un programa executable emmagatzemat al disc consisteix només en codi i dades estàtiques; per posar-lo en execució, el SO necessita:

1. Assignar-li un espai de memòria per al codi, les dades i la pila (*stack*).
2. Inicialitzar els registres de la CPU (incloent-hi el comptador de programa o *Program Counter*).
3. Donar-li accés als dispositius d'entrada/sortida en mode kernel.

El Process Control Block (PCB)

Per gestionar tota la informació associada a cada procés, el nucli del sistema utilitza una estructura de dades anomenada **PCB**. El PCB es divideix principalment en tres blocs:

- **Espai d'adreces**: Descripció i rang de memòria de les regions de codi, dades i pila.
- **Context Software (SW)**: Identificador del procés (**PID**), PID del pare (**PPID**), estat del procés, credencials de l'usuari/grup, informació per a la planificació, informació de gestió d'E/S, comptabilitat de recursos (*accounting*) i la taula de programació de senyals.
- **Context Hardware (HW)**: Registres de la CPU salvats (Program Counter, registres generals) i la taula de pàgines per a la gestió de memòria.

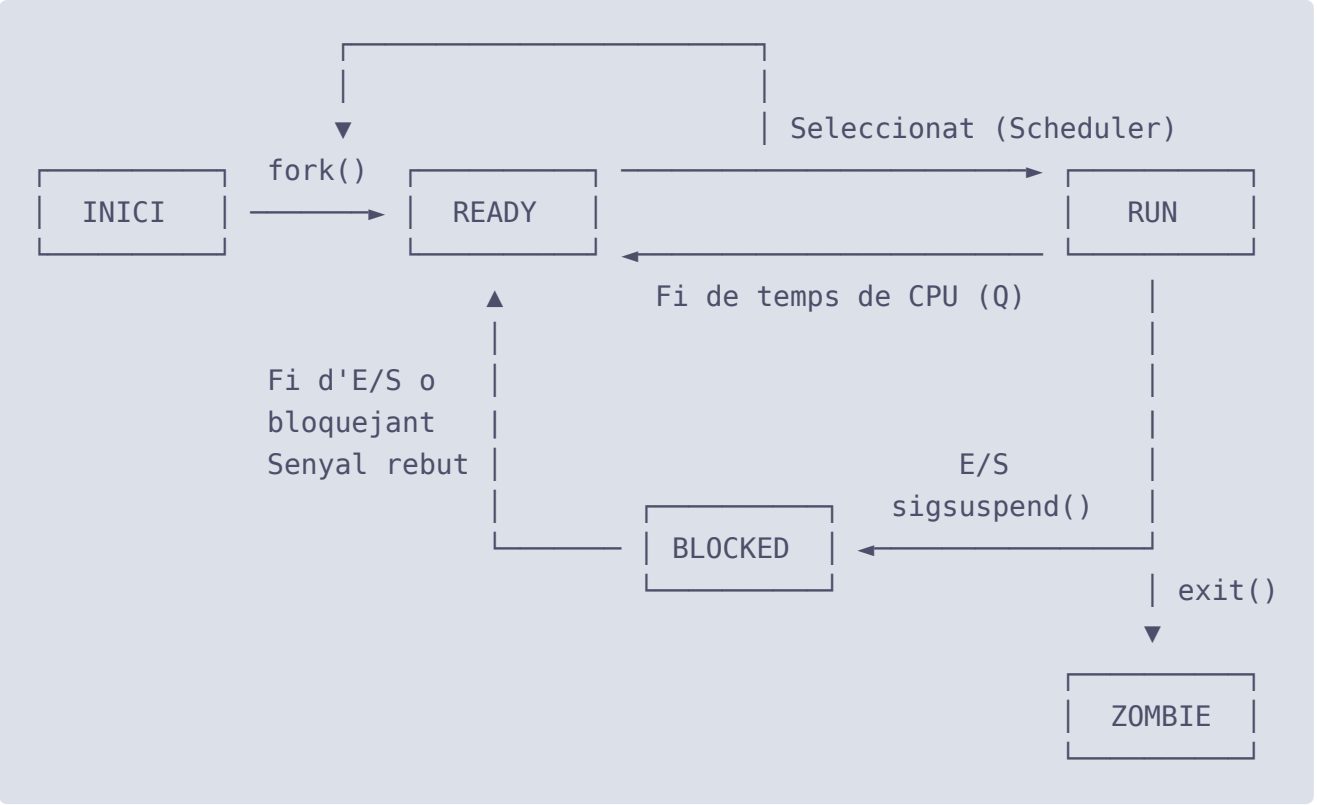
---

2. Concurrència, Paral·lelisme i Estats dels Processos

- **Concurrència**: Capacitat d'executar diversos processos de manera simultània compartint la mateixa CPU. Si el SO va alternant ràpidament entre els processos creant la il·lusió d'execució simultània, es parla de **paral·lelisme virtual**.
- **Paral·lelisme**: Execució simultània real de diferents processos en múltiples nuclis o CPUs hardware.
- **Processos Seqüencials**: Execució ordenada un rere l'altre, on el programador força la sincronització (per exemple, col·locant un `waitpid` o mitjançant senyals).

Graf d'Estats del Procés

Atès que un procés no sempre està utilitzant la CPU (sovint espera dispositius o altres processos), el SO el classifica en diferents **estats** gestionats mitjançant cues o llistes:



1. `RUN`: El procés té assignada la CPU i n'està executant instruccions.
2. `READY`: El procés està preparat per executar-se i s'espera a la cua fins que el planificador li assigni la CPU.
3. `BLOCKED` / `WAITING`: El procés ha realitzat una crida d'E/S bloquejant (`read`, `write`, `open`) o està esperant un esdeveniment/senyal (`sigsuspend`) i ha alliberat la CPU.
4. `ZOMBIE`: El procés ha finalitzat la seva execució (`exit`) i ha alliberat els seus recursos de memòria, però el seu PCB es conserva al nucli guardant l'*exit code* fins que el procés pare el llegeixi amb `waitpid`.

---

3. Crides a Sistema per a la Gestió de Processos (en C)

`fork()` — Creació de Processos

La crida a sistema `fork()` crea un nou procés que és una **duplicació o clon de l'espai d'adreces del pare**.

- **Valor de retorn**:
  - Al procés **fill** li retorna `0`.
  - Al procés **pare** li retorna el **PID del fill** creat.
  - Retorna `-1` si hi ha un error.
- **Herència**: El fill hereta una còpia privada de l'espai d'adreces lògic (codi, dades, pila), les credencials, les variables d'entorn i la taula d'accions de senyals. No hereta el PID, el PPID, els comptadors d'ús ni els senyals pendents/alarmes.

`execlp()` — Canvi d'Executable (Mutació)

La crida a sistema `execlp()` fa que el procés **muti**: reemplaça tot el contingut del seu espai d'adreces (codi, dades i pila) pel d'un nou fitxer executable.

- Manteniment d'identitat: El **PID es conserva**, així com els fitxers oberts i les variables d'entorn.
- Si la crida té èxit, **no retorna mai** al codi original ja que el seu codi s'ha esborrat de la RAM. Si retorna `-1`, és que l'executable no s'ha trobat o ha fallat.

`exit()` i `waitpid()` — Terminació i Sincronització

- `exit(code)`: Finalitza el procés voluntàriament, allibera la memòria i emmagatzema el valor `code` (*exit code*) al PCB, passant a estat ZOMBIE.
- `waitpid(pid, &status, options)`: Bloqueja el pare fins que el fill finalitza.
  - Passar `pid = -1` fa que s'esperi a qualsevol dels seus fills.
  - Recupera l'*exit code* del PCB del fill i **allibera finalment la memòria del PCB**.

Exemple de Codi 1: Esquema de creació concurrent de N fills

En aquest patró crea (N) processos fills que s'executen de forma **concurrent** i posteriorment el pare espera la finalització de tots ells utilitzant `waitpid`:

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>

#define NUM_PROCS 3

int main(void) {
    char buf;
    int i, ret;
    int status;
    int pid_wait;

    for (i = 0; i < NUM_PROCS; i++) {
        ret = fork();

        if (ret < 0) {
            perror("Error en executar fork");
            exit(EXIT_FAILURE);
        }

        if (ret == 0) {
            sprintf(buf, "[FILL] Sóc el fill %d amb PID %d (PPID %d)\n",
i, getpid(), getppid());
            write(1, buf, strlen(buf));
```

```
            exit(i + 1);
        }
    }

    while ((pid_wait = waitpid(-1, &status, 0)) > 0) {
        if (WIFEXITED(status)) {
            sprintf(buf, "[PARE] El fill amb PID %d ha acabat amb exit
code: %d\n", pid_wait, WEXITSTATUS(status));
            write(1, buf, strlen(buf));
        }
    }

    return 0;
}
```

Exemple de Codi 2: Creació d'un fill i mutació amb `execlp`

Aquest patró mostra com una *shell* executa comandes externes (per exemple, `ls -l`) realitzant un `fork()`, carregant el nou executable amb `execlp()` i esperant que acabi:

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>

int main(void) {
    char buf;
    int ret = fork();

    if (ret < 0) {
        perror("Error en executar la crida fork");
        exit(EXIT_FAILURE);
    }

    if (ret == 0) {
        execlp("ls", "ls", "-l", (char *)NULL);

        perror("Error en executar execlp");
        exit(1);
    }
    else {
        waitpid(ret, NULL, 0);
        sprintf(buf, "[PARE] La comanda 'ls -l' executada pel fill ha
finalitzat.\n");
        write(1, buf, strlen(buf));
```

```
    }

    return 0;
}
```

---

4. Comunicació i Sincronització amb Senyals (*Signals*)

Els **senyals** són mecanismes d'interrupció per software que permeten a un procés o al propi nucli notificar un esdeveniment asíncron a un altre procés.

Principals crides a sistema de senyals:

- `kill(pid, signum)`: Envia el senyal `signum` (ex: `SIGUSR1`, `SIGKILL`) al procés destinatari `pid`.
- `sigaction(signum, &tractament, &antic)`: Reprograma l'acció a realitzar en rebre un senyal concret.
- `sigprocmask(mode, &mask, NULL)`: Bloqueja o desbloqueja la recepció dels senyals especificats a la màscara.
- `sigsuspend(&mask)`: Bloqueja el procés d'una manera segura en un únic pas atòmic, canviant la màscara de senyals i esperant la recepció d'un senyal no bloquejat per despertar-se.
- `alarm(segons)`: Programa un temporitzador al nucli per rebre un senyal `SIGALRM` automàticament passats (N) segons.

Espera Activa vs. Bloqueig de Procés

A l'hora d'esperar un esdeveniment o senyal d'un altre procés, hi ha dues alternatives:

1. **Espera Activa (`while (!rebut);`)**: El procés comprova contínuament una variable en un bucle consumint el 100% de la CPU. Només es recomana si el temps d'espera és extremadament curt.
2. **Bloqueig (`sigsuspend(&mask)`)**: El procés surt de l'estat `RUN` i s'espera en estat bloquejat, alliberant la CPU per a la resta de processos de la màquina. És la forma recomanada per a esperes mitjanes o llargues.
   Aquí tens els **2 exemples petits i concrets** que demanes, seguint l'estil exacte de laboratori (`char buf`, `sprintf`, `strlen` i `write`):

Exemple de Codi 3: `WIFSIGNALED` + `WTERMSIG` i creació de màscara `SIGALRM`

Aquest exemple mostra com bloquejar el senyal `SIGALRM` amb `sigprocmask` i com el pare detecta si un procés fill ha finalitzat de manera involuntària a causa d'un senyal utilitzant `WIFSIGNALED` i n'obté el número amb `WTERMSIG`:

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <signal.h>

int main(void) {
    char buf[80];
    int status;
    sigset_t mask;

    // Creació i configuració d'una màscara per bloquejar SIGALRM
    sigemptyset(&mask);
    sigaddset(&mask, SIGALRM);
    sigprocmask(SIG_BLOCK, &mask, NULL);

    int pid = fork();

    if (pid == 0) {
        while (1);
    } else {
        kill(pid, SIGKILL);
        waitpid(pid, &status, 0);

        // Comprovem si el fill ha finalitzat per la recepció d'un senyal
        if (WIFSIGNALED(status)) {
            sprintf(buf, "[PARE] El fill %d ha mort pel senyal %d\n", pid,
WTERMSIG(status));
        } else if(WIFEXITED(status))  {
             sprintf(buf, "[PARE] El fill %d ha mort naturalment amb exit:
%d\n", pid, WEXITSTATUS(status));
        }
        write(1, buf, strlen(buf));
    }

    return 0;
}
```

Exemple de codi 4: Creació de `sigalrm_handler` + configuració de màscara

Aquest exemple mostra la definició d'una funció de tractament `sigalrm_handler` i la seva associació al senyal `SIGALRM` mitjançant l'estructura `sigaction` i la seva màscara interna `sa_mask`:

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>

void sigalrm_handler(int s) {
    char buf[80];
    sprintf(buf, "[HANDLER] Alarma rebuda correctament (senyal %d)!\n",
s);
    write(1, buf, strlen(buf));
}

int main(void) {
    struct sigaction sa;
    sigset_t mask;

    // Configuració de la màscara interna i del handler
    sigemptyset(&mask);
    sa.sa_mask = mask;  // Senyals a bloquejar durant l'execució del
handler
    sa.sa_flags = SA_RESTART;
    sa.sa_handler = sigalrm_handler;

    // Associem el handler al senyal SIGALRM
    if (sigaction(SIGALRM, &sa, NULL) < 0) {
        perror("Error en sigaction");
        exit(1);
    }

     sigfillset(&mask);
     sigdelset(&mask, SIGALRM);

    alarm(1);
    sigsuspend(&mask);

    return 0;
}
```

---

5. Gestió Interna del SO, Canvi de Context i Planificació

Organització de Cues al Nucli

El nucli manté els PCB organitzats en diferents estructures de dades (listes i cues) segons el seu estat:

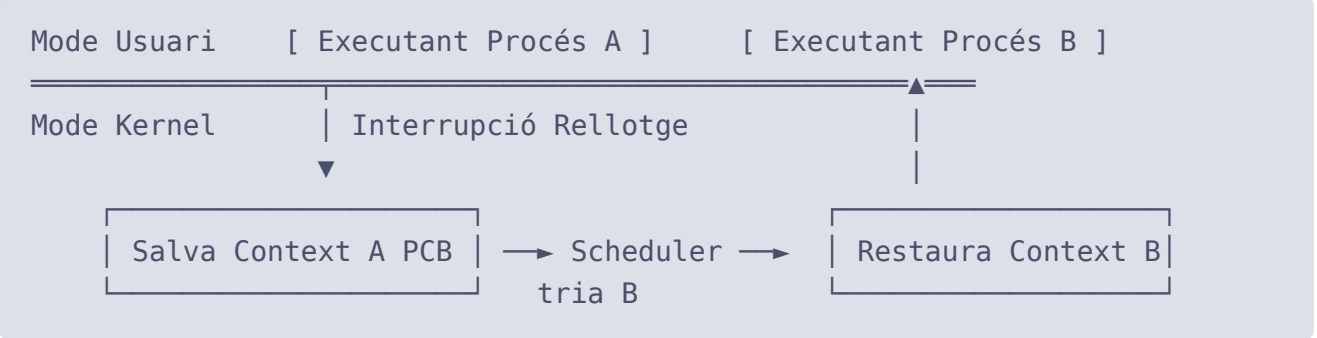
- **Cua de Ready**: Conté tots els processos en estat `READY` esperant per rebre temps de CPU.
- **Cues de Dispositius/Esdeveniments**: Cues de processos bloquejats esperant operacions d'E/S o senyals.

El Canvi de Context (*Context Switch*)

És el mecanisme pel qual la CPU deixa d'executar un procés `A` per passar a executar un procés `B`:

1. Ocorre un esdeveniment (interrupció de rellotge, crida a sistema bloquejant, etc.).
2. Es canvia a **Mode Kernel**.
3. El SO desa el context del procés `A` (registres hardware, PC) al seu PCB.
4. El planificador (*scheduler*) selecciona el procés `B` de la cua de `READY`.
5. Es restaura el context del procés `B` des del seu PCB i es retorna a **Mode Usuari** per continuar l'execució de `B`.

> *El temps del canvi de context no és temps útil d'aplicació (és sobrecàrrega o overhead), per la qual cosa ha de ser extremadament ràpid.*



Polítiques de Planificació

L'algorisme de planificació determina quin procés de la cua de `READY` passa a estat `RUN`:

- **Polítiques Apropiatives (*Preemptive*)**: El SO té la capacitat de treure-li la CPU a un procés abans que acabi (per exemple, quan s'exhaureix el seu temps màxim de processament).
- **Round Robin (RR)**: Algorisme **apropiatiu** basat en una cua FIFO (_First In First Out) de `READY` on cada procés rep la CPU durant un interval màxim anomenat **Quàntum de Temps ((Q))** (típicament entre 10 ms i 100 ms).
  - Si el procés es bloqueja o acaba abans d'exhaurir el quàntum, allibera la CPU.
  - Si exhaureix el quàntum (Q), la **interrupció de rellotge** desallotja el procés, el posa al final de la cua de `READY` i assigna la CPU al següent procés.

6. Pregunta Zhuo. Si un procés pare mor abans que el seu fill finalitzi la seva execució, que pasa?

Quan el procés pare mor abans que el seu fill finalitzi la seva execució, el procés fill passa a anomenar-se **procés orfe** (*orphan process*).

En els sistemes operatius tipus UNIX/Linux, passa el següent:

1. **Adopció per part del procés init**: El procés **init** (o el procés inicial del sistema amb **PID 1**, com `systemd` als Linux moderns) **adopta automàticament el procés orfe**.
2. **Actualització del PPID**: El camp `PPID` (*Parent Process ID*) del procés fill canvia i passa a valer `1` (el PID d' `init`).
3. **Neteja de la memòria PCB**: Quan el fill finalment acaba la seva execució (`exit`), el procés `init` s'encarrega d'executar la crida a sistema `wait()` / `waitpid()` per recollir el seu estat de sortida i alliberar l'estructura **PCB** del nucli, evitant així que el fill es quedi bloquejat indefinidament com a procés **zombie**.